Listing 5.1. Implementation of a spinlock mutex using std::atomic_flag

Listing 5.2. Reading and writing variables from different threads

Listing 5.3. Order of evaluation of arguments to a function call is unspecified

Listing 5.4. Sequential consistency implies a total ordering

Listing 5.5. Relaxed operations have very few ordering requirements

Listing 5.6. Relaxed operations on multiple threads

Listing 5.7. Acquire-release doesn’t imply a total ordering

Listing 5.8. Acquire-release operations can impose ordering on relaxed operations

Listing 5.9. Transitive synchronization using acquire and release ordering

Listing 5.10. Using std::memory_order_consume to synchronize data

Listing 5.11. Reading values from a queue with atomic operations

Listing 5.12. Relaxed operations can be ordered with fences

Listing 5.13. Enforcing ordering on nonatomic operations

Chapter 6. Designing lock-based concurrent data structures

Listing 6.1. A class definition for a thread-safe stack

Listing 6.2. The full class definition for a thread-safe queue using condition variables

Listing 6.3. A thread-safe queue holding std::shared_ptr<> instances

Listing 6.4. A simple single-threaded queue implementation

Listing 6.5. A simple queue with a dummy node

Listing 6.6. A thread-safe queue with fine-grained locking

Listing 6.7. A thread-safe queue with locking and waiting: internals and interface

Listing 6.8. A thread-safe queue with locking and waiting: pushing new values